

小车巡线

小车巡线

1. 快速上手
2. 硬件接线
3. 使用方法
4. 现象结果
5. 代码解释

`line_following_init` 函数

`app_motor_uart` (电机驱动模块)

`grayscale_sensor` (灰度传感器模块)

`trace_task` (巡线任务模块)

`main` (主程序)

1. 快速上手

本教程讲解如何使用 ESP32-S3 主板在获取八路灰度巡线模块的数据之后开始在赛道上巡线，以及如何修改代码中的部分参数，使小车在用户自己的赛道场景中运行的效果更好。

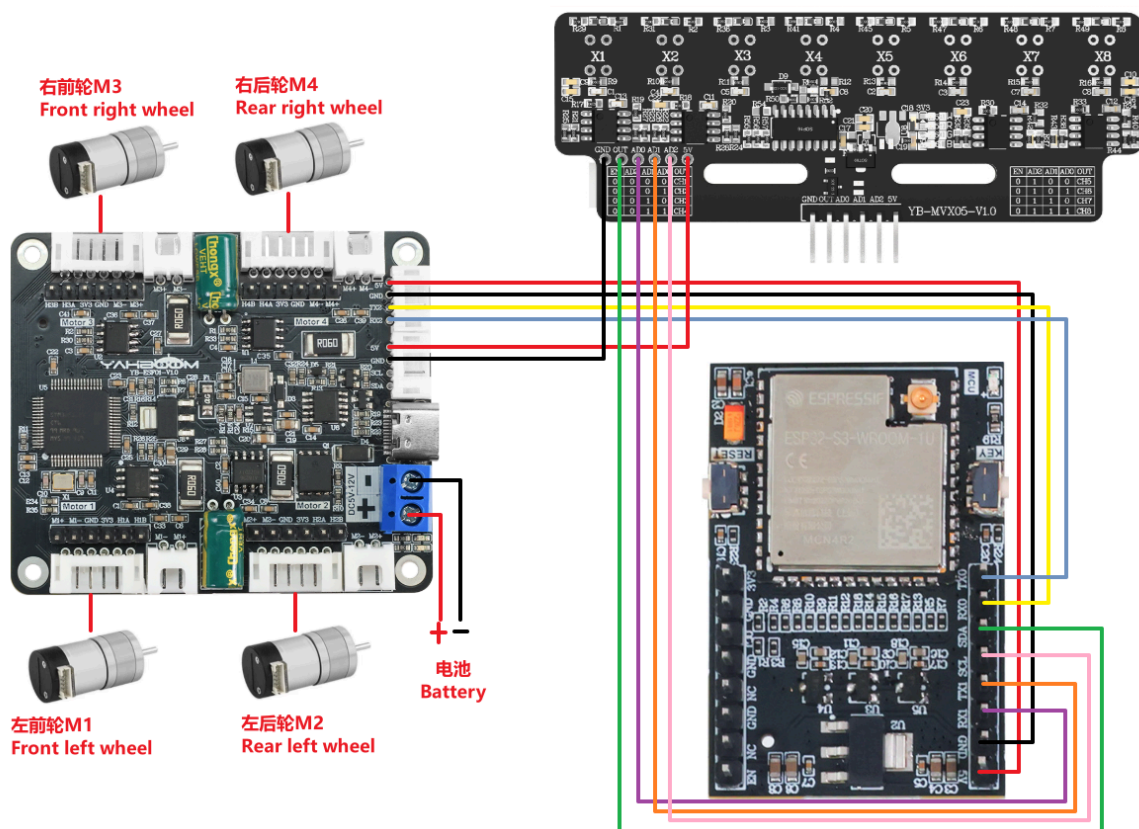
教程中使用的硬件均是亚博售卖的 ESP32-S3 WIFI图传模块lite、八路灰度巡线模块、轮式小车底盘 V1、310电机、四路电机驱动板、7.4V电池组，此代码因为占用了烧录串口，所以需要先对 ESP32-S3 烧录程序，然后再进行接线，就可以上电开始巡线了。如果效果不佳，可以跳转到代码讲解章节，对巡线参数进行修改，优化巡线效果。

除此之外，还需要修改合适的电机PID参数，本案例中使用的是310电机，使用的电机PID值为：P:1.9,I:0.2,D:0.8，使用PC串口助手与四路电机驱动板通信，发送指令修改PID值，如果使用的是其他电机，需要自行测试出一个合适的电机PID参数。

此调优步骤需要用户具备一定的调试小车经验。非亚博的模块仅供参考。

2. 硬件接线

由于本案例使用的ESP32只有一个5V引脚，因此需要将八路灰度模块的5V接入到四路电机驱动板上。



八路灰度巡线模块	ESP32-S3
AD0	RX1(35)
AD1	TX1(36)
AD2	SCL(37)
OUT	SDA(38)

ESP32-S3	四路电机驱动板
5V	5V
GND	GND
TX0(43)	RX2
RX0(44)	TX2

八路灰度巡线模块	四路电机驱动板
5V	5V
GND	GND

这里的串口和IIC标识只是使用的ESP32模块板子上的标识，实际上只有与四路电机驱动板相连的引脚使用到了串口功能。

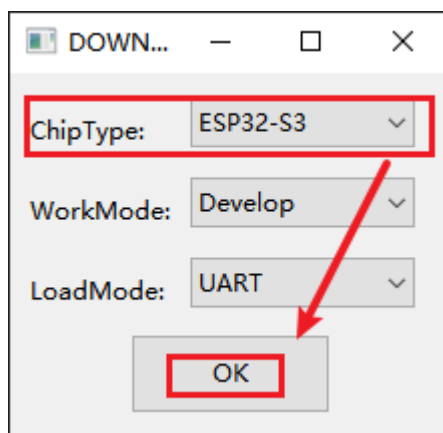
亚博售卖的八路灰度模块连接使用的线材为XH2.54转6pin杜邦线：



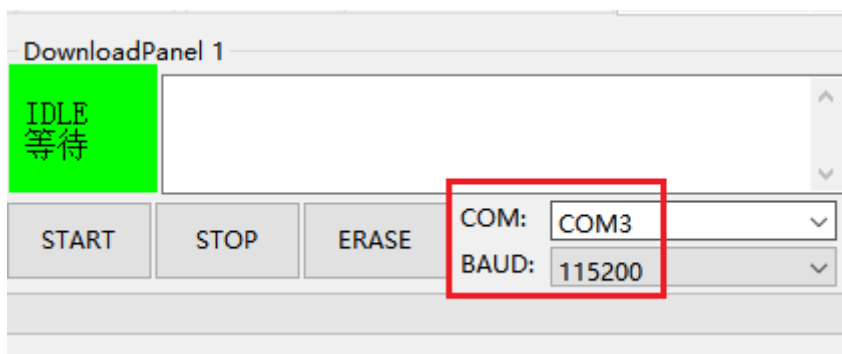
亚博售卖的四路电机驱动板与310电机的连接方法可以到四路电机驱动板资料中找到《常用电机介绍以及用法》的教程，查看详细的具体接线方式，由于篇幅较长，这里不再展开。

3. 使用方法

1. 打开烧录工具 `flash_download_tool`，芯片选择ESP32-S3，然后就点击OK。

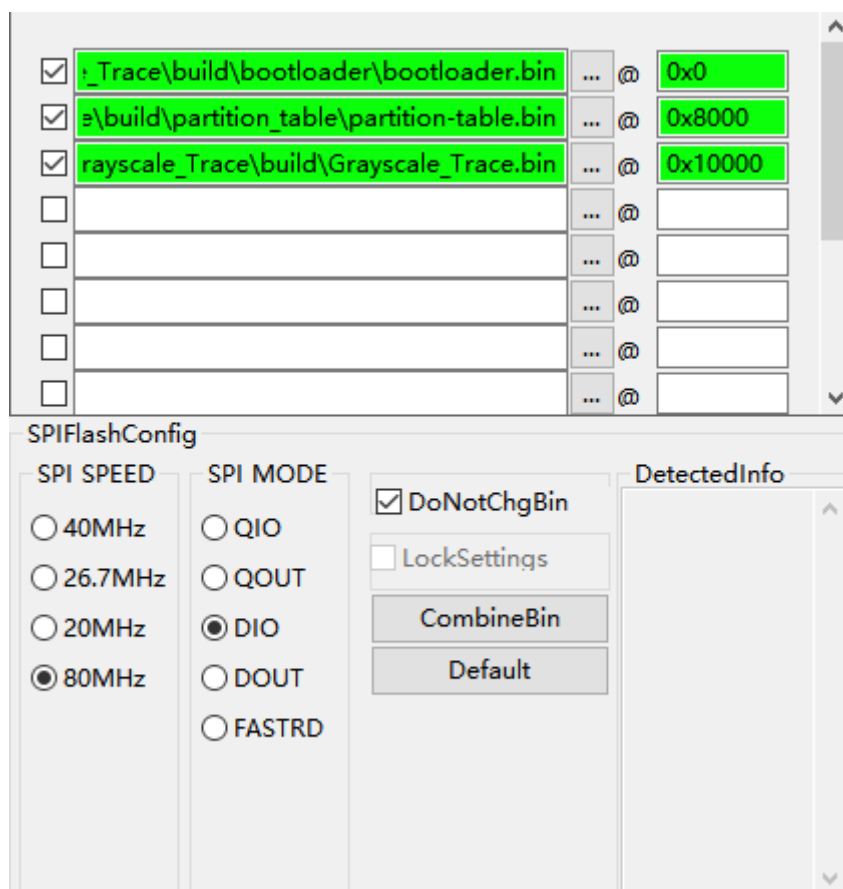


2. 通过数据线将 ESP32-S3 连接到电脑，然后软件里面选择好COM口和波特率。



3. 在 `SPIDownload` 窗口选择三个要烧录的ESP32S3的固件，路径一般在代码工程根目录下的 `build` 文件夹里，格式为bin文件，固件地址记得输入，必须确认勾选bin文件左边的方框，并且文件和地址都为绿色背景。

勾选DoNotChgBin，其他配置保持默认即可。

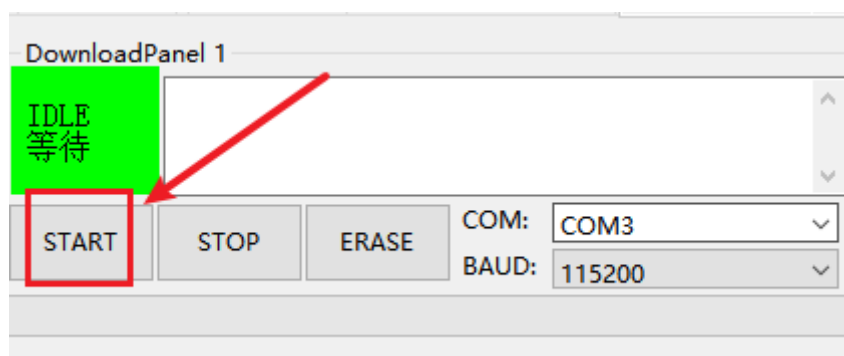


文件与地址对应关系如下表所示：

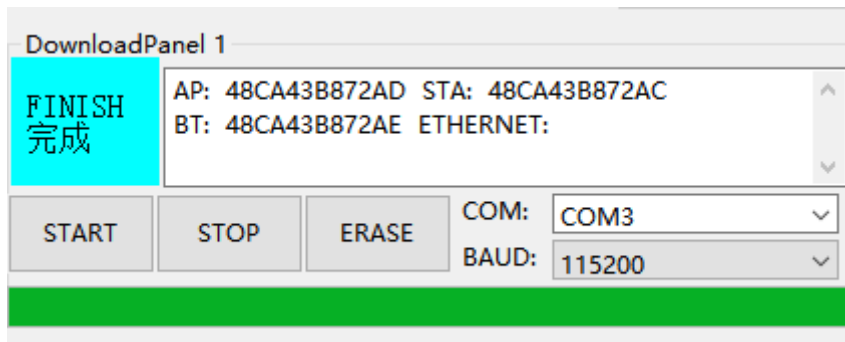
固件名称	固件地址	备注
bootloader.bin	0x0000	引导文件
partition-table.bin	0x8000	分区表文件
Grayscale_Trace.bin	0x10000	功能文件

4. 点击START按钮，工具即自动开始烧录固件。

注：如果没有自动开始烧录固件，请先按住IO0键，再按EN键，然后同时松开，手动进入烧录模式，然后再点击START按钮。



5. 下载完成后，提示蓝色FINISH标识。此时断电重启单片机或者按一下EN键重启程序，关闭烧录工具。最后使用串口助手打开此端口就可以看到打印信息了。



4. 现象结果

上电之后，程序开始运行。此时如果八路灰度巡线模块上的八个LED灯都是全亮或者全灭，小车则不会动，防止因为被放在了错误的地图上乱跑。当巡线模块上的灯不一致时，小车开始巡线。

在要巡的线上的巡线模块LED灯，需要和线外的赛道相反，也就是说假如线上的LED灯是灭的，线外面的赛道上对应的LED灯需要亮起，才能正常巡线。

5. 代码解释

```
//#####  
// 重要配置参数 Important Configuration Parameters  
//#####  
// 根据自己使用的电机类型，选择对应的电机参数配置  
// Select the corresponding motor parameter configuration according to the type  
of motor you use  
#define MOTOR_TYPE 2 //1:520电机 2:310电机 3:测速码盘TT电机 4:TT直流减速电机 5:L型520  
电机  
//1:520 motor 2:310 motor 3:speed code disc TT motor 4:TT DC  
reduction motor 5:L type 520 motor  
//#####  
// 将灰度巡线模块放置于赛道，观察正对着线的灯，如果灯亮，则数值为1，LINE_RAW_VALUE=1；如果灯  
灭，则数值为0，LINE_RAW_VALUE=0  
// Place the grayscale line-following module on the track and observe the light  
facing the line. If the light is on, the value is 1, LINE_RAW_VALUE=1; if the  
light is off, the value is 0, LINE_RAW_VALUE=0  
#define LINE_RAW_VALUE 1  
//#####
```

代码开头的地方有 `MOTOR_TYPE` 和 `LINE_RAW_VALUE` 两个宏定义设置。

- `MOTOR_TYPE`：根据自己所购买的电机修改MOTOR_TYPE，代码中适配的电机都是亚博目前在售的电机。
- `LINE_RAW_VALUE`：需要将灰度巡线模块放置于赛道，观察正对着线的灯，如果灯亮，则填LINE_RAW_VALUE=1；如果灯灭，则数值为0，LINE_RAW_VALUE=0。

```
// Grayscale_Trace.c  
void line_following_init(line_following_t* controller) {  
    controller->kp = 270.0f; // 比例系数  
    controller->ki = 0.5f; // 积分系数  
    controller->kd = 50.0f; // 微分系数
```

```

controller->base_speed = 450;    // 基础速度
controller->max_speed = 800;    // 最大速度

controller->sensor_weights[0] = -5.0f;
controller->sensor_weights[1] = -4.0f;
controller->sensor_weights[2] = -2.0f;
controller->sensor_weights[3] = -1.0f;
controller->sensor_weights[4] = 1.0f;
controller->sensor_weights[5] = 2.0f;
controller->sensor_weights[6] = 4.0f;
controller->sensor_weights[7] = 5.0f;
//...
}

```

line_following_init 函数

优化巡线效果重点修改项：

PID 参数

- **kp**: 调大 **kp** 可使响应更快，但易震荡；调小 **kp** 可减少震荡，但响应会变慢且可能偏离路线。
- **ki**: 调大 **ki** 可更快消除稳态误差，但易引起超调和震荡；调小 **ki** 可降低超调风险，但消除误差变慢甚至无法完全消除。
- **kd**: 调大 **kd** 可提升系统稳定性，减少摆动和超调，但过大会对噪声敏感并引发抖动；调小 **kd** 可降低对噪声的敏感，但抑制摆动能力减弱，响应可能滞后或超调。

速度参数

- **base_speed**: 决定小车在平稳巡线时的前进速度，影响整体效率和弯道响应。
- **max_speed**: 限制小车轮的最高转速，防止小车加速偏差过大、过快。

灰度巡线模块权重参数

- **sensor_weights**: 修改灰度巡线模块上的权重，第一个值指代模块最边缘的X1灯，第二个值为X2灯。第一个数值调大，则在X1灯亮时反应更剧烈，调小则平缓。

```

// app_motor_uart.c
void Set_Motor(int MOTOR_TYPE)
{
    if(MOTOR_TYPE == 1)
    {
        send_motor_type(1);
        //...
    }
    //...
}

void Contrl_Speed(int16_t M1_speed,int16_t M2_speed,int16_t M3_speed,int16_t
M4_speed)
{

sprintf((char*)send_buff, "$spd:%d,%d,%d,%d#", M1_speed,M2_speed,M3_speed,M4_speed)
;

    Send_Motor_ArrayU8(send_buff, strlen((char*)send_buff));
}

```

```
}
```

app_motor_uart (电机驱动模块)

- `Set_Motor`: 根据预设的电机类型 (MOTOR_TYPE), 配置电机的相关参数 (类型、减速比、磁环线、轮径、死区)。
- `Contrl_Speed`: 通过UART向电机驱动板发送指令, 以编码器闭环控制四个电机的速度。

```
// grayscale_sensor.c
void Grayscale_Sensor_Read_All(uint16_t* sensor_values)
{
    uint8_t i;
    for (i = 0; i < GRAYSCALE_SENSOR_CHANNELS; i++)
    {
        _select_channel(i);
        _delay_us(50);
        sensor_values[i] = Read_OUT_value();
    }
}
```

grayscale_sensor (灰度传感器模块)

- `Grayscale_Sensor_Read_All`: 依次选择所有传感器通道, 读取并返回每个通道的输出值列表。

```
// trace_task.c
bool check_sensors_safe(line_following_t* controller, uint16_t* sensor_values) {
    uint16_t first_value = sensor_values[0];
    //...
    return false;
}

float calculate_error(line_following_t* controller, uint16_t* sensor_values,
uint16_t line_raw_value) {
    float weighted_sum = 0.0f;
    int active_sensors = 0;
    //...
    return error;
}

float pid_control(line_following_t* controller, float error) {
    if (fabsf(error) < 0.6f) {
        error = 0.0f;
    }
    //...
    return output;
}

void differential_speed_control(line_following_t* controller, float pid_output,
                                int16_t* left_speed, int16_t* right_speed) {
    //...
```

```

}

void follow_line(line_following_t* controller, uint16_t* sensor_values,
                uint16_t line_raw_value) {
    if (controller->motor_locked) {
        if (check_sensors_safe(controller, sensor_values)) {
            controller->motor_locked = false;
        } else {
            Contrl_Speed(0, 0, 0, 0);
            return;
        }
    }

    float error = calculate_error(controller, sensor_values, line_raw_value);
    float pid_output = pid_control(controller, error);
    int16_t left_speed, right_speed;
    differential_speed_control(controller, pid_output, &left_speed,
                              &right_speed);

    Contrl_Speed(left_speed, left_speed, right_speed, right_speed);
}

```

trace_task (巡线任务模块)

- `check_sensors_safe` : 检查所有灰度传感器是否都显示相同值, 用于在启动时判断小车是否处于安全状态。
- `calculate_error` : 根据灰度传感器读取的值和预设权重, 计算小车偏离线中心的误差值。
- `pid_control` : 执行PID控制算法, 根据输入的误差值计算控制输出。
- `differential_speed_control` : 根据PID控制输出, 计算左右轮的差速, 从而调整小车转向。
- `follow_line` : 巡线主函数, 负责读取传感器、执行安全检查、计算误差、进行PID控制和差速控制来驱动电机。

```

// Grayscale_Trace.c
void app_main(void)
{
    uart0_init();
    Grayscale_Sensor_Init();
    Set_Motor(MOTOR_TYPE);
    send_motor_PID(1.9, 0.2, 0.8);
    line_following_init(&g_line_controller);

    xTaskCreate(Grayscale_ReadData_Task, "GrayscaleReadData", 4096, NULL, 3,
                NULL);
    xTaskCreate(Trace_Task, "Trace", 4096, NULL, 4, NULL);
}

```


main (主程序)

- `app_main`: 程序入口。依次初始化了UART、灰度传感器、电机参数和巡线控制器。然后创建了两个独立的FreeRTOS任务: `Grayscale_ReadData_Task` 用于持续读取传感器数据, `Trace_Task` 用于执行巡线逻辑。这种多任务架构使得传感器数据采集和巡线控制可以并行处理, 互不阻塞。